
Should I train or should I grow?

Single-Run Architecture Search by Network Growth

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Finding an appropriate architecture for a specific task is a complex process of
2 interwoven optimization and design choices. Baseline architectures are the result
3 of tremendous research effort focusing on carefully prepared tasks. On new and
4 unseen datasets, it is common either to start with large architectures and reduce
5 them afterwards, or to use Neural Architecture Search techniques. We propose
6 a third and under-explored way: start from a small seed model and grow its
7 computational graph by adding neurons and layers, in a single training run. Based
8 on functional analysis, we estimate when and where we lack expressivity and how
9 much to extend the architecture to mitigate it. Given a task, our method Demeter
10 is anytime: it can be stopped at any point along the Pareto front of accuracy and
11 complexity, yielding a model that is not over-parameterized but instead “just large
12 enough”. Experimental results on multiple datasets demonstrate that Demeter is
13 competitive in terms of accuracy while keeping training time and model size low.

14 1 Introduction

15 The performance of deep neural networks depends critically on architectural choices: depth, width,
16 connectivity patterns, operators, skip connections, and the allocation of parameters across layers all
17 influence both optimization and generalization. Neural Architecture Search (NAS) addresses this
18 issue by automating the design of neural networks, replacing manual architecture engineering with a
19 search procedure over a predefined search space. Early NAS methods based on reinforcement learning
20 or evolutionary algorithms demonstrated that competitive architectures could indeed be discovered
21 automatically, but often at the cost of training a large number of candidate networks [1–3]. This
22 computational burden has motivated a long line of work on making NAS more efficient, including
23 cell-based search spaces [4, 5], weight-sharing supernet [6], differentiable [7–9], and zero-cost
24 proxies [10].

25 This paper explores an alternative perspective: instead of searching over many architectures, one
26 may start from a small model and progressively grow it during training. Neural architecture growth
27 amortizes architectural exploration along a single optimization trajectory. The neural network is
28 trained, analyzed, and expanded only when its current architecture becomes insufficient. This avoids
29 repeatedly training independent prototypes, while preserving the weights and representations already
30 learned by the model. Moreover, the exploration benefits from the guidance of the functional gradient,
31 instead of sampling the space of architectures randomly.

32 Early work on neural network growth focused on appropriately adjusting the size of a fixed family of
33 architectures, *i.e.* adjusting width or depth in ResNets. More generally, an architecture can be viewed
34 as its computational graph, which is necessarily a Directed Acyclic Graph (DAG). In this work, we
35 extend the ideas introduced in [11, 12] to grow convolutional DAG networks with arbitrary topology,
36 substantially increasing the model search space over previous growing methods.

Demeter uses a first-order principled criterion to identify useful (functional) directions that the current architecture cannot represent via gradient descent and a rule for adding parameters to compensate for this lack of expressivity.

The goal of this paper is to show that such growth-based methods can provide a competitive alternative to state-of-the-art NAS techniques, while requiring fewer architectural trials and a lower computational budget as shown on Figure 1. This makes neural growth a natural candidate for ‘anytime’ architecture design, especially when the cost of search is a limiting factor. Our novel approach is indeed able to yield the best architecture design at anytime during training, with increasing architecture size and accuracy as time goes by. We name our approach Demeter after the Greek goddess of agricultural growth.

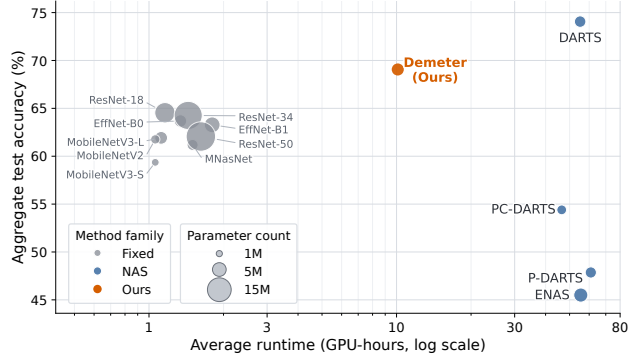


Figure 1: Aggregate test accuracy against runtime on the Unseen Data Challenge [13], marker area being proportional to the average parameter count. Demeter offers an unconstrained architecture option at much reduced compute cost compared to large supernetworks (DARTS), while being much more flexible by nature compared to fixed-family architectures (ResNets, MobileNets).

Contributions:

- We introduce the first convolutional free-form DAG growth method for various datasets, achieving results comparable to SOTA NAS methods.
- We introduce new schemes for growing networks which can scale to the combinatorial complexity of a DAG, including criteria for when, where, and how much to grow.
- We demonstrate the ‘anytime’ nature of our method, as an amortized alternative to multi-trial NAS procedures.
- We show that our method outperformed NAS methods on several datasets from NAS benchmarks (including atypical datasets for which suitable architectures are expected to be unusual), while providing a comparison with fixed architectures baselines.

2 Related Work

Benchmark models Standard fixed-size architecture families that perform strongly for image classification tasks include ResNet [14], WideResNet [15] architectures. The MobileNetV2 [16], MobileNetV3 [17], and EfficientNet [18] architectures are optimized to achieve a good tradeoff between accuracy and complexity (e.g., parameter count or inference latency).

Neural Architecture Search (NAS). The original Neural Architecture Search methods are based on evolutionary algorithms [2, 3] and Reinforcement Learning [1]. To reduce the computational complexity, later works [4, 5] reduce the search space to a single cell encoded as a Directed Acyclic Graph. This cell is then inserted into a fixed macro architecture that consists of an initial convolutional layer, followed by a stack of, e.g., 3 cells, followed by a downsampling layer. This stack of cells and downsampling is repeated multiple times to achieve the required network depth, finally followed by one or more fully-connected layers. We also use a cell-based architecture, considering each resolution as a cell (see Fig. 4). However, unlike these previous works, our cells are independent and do not need to be identical. Most NAS methods optimize a single objective, such as accuracy. In order to traverse the Pareto front between accuracy and complexity [19] introduced an evolutionary algorithm for multi-objective architecture search.

Supernetworks and Differential NAS. To amortize the cost of training many networks from scratch, ENAS [6] samples a subnetwork from a large supernetwork and trains it without resetting

the weights. DARTS [7] relaxes the discrete optimization problem to a continuous one, where we instead consider a softmax distribution over operator weights for every possible connection in the supernet. The operator weights are optimized by gradient descent at the same time as the supernet weights, resulting in a “one-shot” NAS method, as the architecture is selected at the end of a single training step. However, the supernet is typically too large to fit in GPU memory, so search is performed on a proxy architecture with reduced depth, where one only searches for a normal and a reduction cell that is repeated throughout the supernet. At the end of the search phase, the found cells are used in the full-depth supernet, which is trained from scratch for the final performance. The use of a proxy architecture introduces an optimization-evaluation gap, and several methods have been proposed to fix this. P-DARTS [8] progressively increases the depth over time, and PC-DARTS [9] stochastically samples a subset of the channel connections. Another issue of supernet-based approaches is that, by design, they set a maximum architecture size, thus preventing the consideration of larger or deeper architectures if needed. On the other hand, since they always train the whole maximum architecture, computational resources will be wasted if the optimal architecture is small.

Training acceleration An alternative approach is to focus on training dynamics by adding neurons to reduce the loss as fast as possible, as advocated by GradMax [20]. This approach adds neurons by maximizing the gradients of the new weights. Along similar lines, Net2Net [21] focuses on accelerating the transfer of knowledge from one architecture to another by splitting existing neurons to preserve the function output. While those methods mention growth, their objective is solely to focus on improving training dynamics to save time, and require defining a target architecture; as they are not NAS techniques, they are not considered in the scope of this study.

Growing Neural Networks. Rather than starting with a large supernet and searching for the optimal subnetwork, growing neural networks start with a small seed network and progressively increase its size. These approaches focus on how to initialize new weights and modify existing weights when the architecture grows. Most efficient methods rely on maximizing the natural gradient of the new weight [11, 22]. These techniques, however, require a target architecture that they aspire to reach by growing. For this reason, they do not ‘discover’ an architecture for the task, but instead work in a reverse compression manner while the target architecture is already known. An exception is LEMONADE [23], which discovers architectures without a predefined target by applying function-preserving network morphisms within an evolutionary search loop, though at the cost of the sample inefficiency typical of population-based search. Building upon the natural gradient approach of TINY [11], [12] introduced the use of directed acyclic graphs to represent the model and added completely new layers to the computational graph. However, the implementation was limited to fully-connected layers and provided a narrow experimental setup. Our work implements convolutional layers and revisits the functional natural gradient approaches, but proposes a principled approach for growing strategies and thorough experiments to compare with state-of-the-art methods in architecture search. Indeed, it yields the best performing architecture at anytime, offering a unique ability to train and grow practical models while limiting the computing resources needed.

3 Method

We represent the network as a Directed Acyclic Graph (DAG): nodes represent a hidden state a_l , while edges represent layer weight matrices (or convolution kernels). Suppose we want to expand the representational power of the network. At any given node l we can:

- (i) Increase the size of any of the nodes already connected to l , by adding new neurons to its input and output layers (increase width or channel number).
- (ii) Add a new node and the corresponding 2 edges to connect any previous node and l (thus add 2 layers).

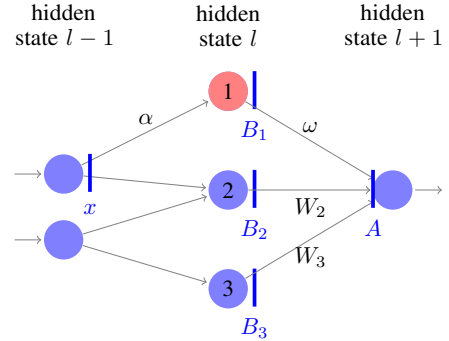


Figure 2: Example of DAG Network. We assume a new node added in color red with new weights α and ω . We define pre-activities as A and post-activities as B or x . [12]

(iii) Add a direct edge from one of the previous nodes to l (add 1 layer).

Choosing which node to grow and how to grow it presents a challenging combinatorial problem. To answer this, we utilize the concept of an expressivity bottleneck, described in the next section.

3.1 Expressivity bottleneck — a functional analysis viewpoint

Figure 3 presents the concept of an *expressivity bottleneck*, introduced by [11]. The manifold $\mathcal{F}_{\mathcal{A}} := \{f_{\theta} \mid \theta \in \Theta_{\mathcal{A}}\}$ represents the functional space parameterized by a neural network, *i.e.* the set of all possible functions one can represent by instantiating parameters of a fixed architecture $\mathcal{A}$. The tangent space at f_{θ} , namely $\mathcal{T}_{\mathcal{A}} := \left\{ \frac{\partial f_{\theta}}{\partial \theta} \delta \theta \mid \text{s.t. } \delta \theta \in \Theta_{\mathcal{A}} \right\}$, consists of all the possible functions one can reach within the current manifold $\mathcal{F}_{\mathcal{A}}$, using small parameter updates, *e.g.* by gradient descent. Now, let us denote by v_{∇} the desired update for the function f_{θ} when we are not constrained by the current architecture:

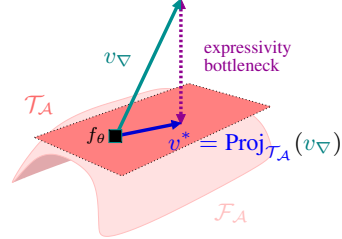


Figure 3: Expressivity bottleneck defined as $v_{\perp} := v_{\nabla} - v^*$. [11]

$$v_{\nabla}(x) := -\nabla_{f_{\theta}(x)} \mathcal{L}(f_{\theta}(x), y(x)) := -\nabla_a \mathcal{L}(a, y(x)) \big|_{a=f_{\theta}(x)} . \quad (1)$$

This is the functional gradient, *i.e.* the gradient of the loss w.r.t. the output of the network. However, in reality, we are constrained by the parameterization of our function. The best update we can perform with the current architecture $\mathcal{A}$ is the projection v^* of that desired update onto the tangent space $\mathcal{T}_{\mathcal{A}}$. Geometrically, in the high-dimensional space of functions, this yields an orthogonal triangle whose hypotenuse v_{∇} is the desired update direction, and the vertical component $v_{\perp}$ (orthogonal to $\mathcal{T}_{\mathcal{A}}$) results from the inability of the architecture to completely express the direction of the loss. This *residual* is defined as:

$$v_{\perp} := v_{\nabla} - v^* \quad (2)$$

$$\text{where } v^* := \text{Proj}_{\mathcal{T}_{\mathcal{A}}}(v_{\nabla}) := \arg \min_{v \in \mathcal{T}_{\mathcal{A}}} \mathbb{E}_{(x,y) \sim \mathcal{P}} [\|v_{\nabla}(x) - v(x)\|_F^2] . \quad (3)$$

The residual's norm $\Psi := \|v_{\perp}\|$ is called the *expressivity bottleneck* of the architecture.

These calculations require only one forward-backward pass, thus moderate overhead, although the corresponding batch size may need to be larger than usual to obtain sufficiently reliable activity statistics.

3.2 Adding neurons

Neurons are added during training so as to minimize the expressivity bottleneck, in order to follow the functional gradient descent more closely. To do this, the bottlenecks are computed not only for the whole function but also for each layer of the network, by considering $v_{\nabla}^l = -\nabla_a \mathcal{L}(f_{l \rightarrow L}(a), y(x)) \big|_{a=a_l(x)}$ where a_l denotes the activities at layer l and $f_{l \rightarrow L}$ denotes the remaining part of the network.

Let us consider k new neurons, of input weights α and output weights ω , to be added to an existing layer l . The resulting change of activities at layer l for sample x can be written as $\delta a_l(x) = \omega \sigma(\alpha a_{l-1}(x) + b)$. To minimize the expressivity bottleneck, this change δa_l should be made as close as possible to the layer residual $v_{\perp}^l$. Unlike TINY and [12], which aim at minimizing $\|\omega \sigma(\alpha a_{l-1} + b) - v_{\perp}^l\|^2$, we propose to only align their directions to stabilize the results, *i.e.* optimizing $\left\langle \frac{\omega \sigma(\alpha a_{l-1} + b)}{\|\omega \sigma(\alpha a_{l-1} + b)\|} \mid \frac{v_{\perp}}{\|v_{\perp}\|} \right\rangle$.

3.3 Adding layers to the DAG

Expanding nodes or adding new ones is not fundamentally different, once we choose the new input and output edges. Since adding a new node is equivalent to expanding an empty layer, the optimization framework described in the previous section still holds. The peculiarity of case (i) is that the best



Figure 4: Macro architecture for Demeter: four growable DAG blocks, one per resolution, each separated by a downsampling layer, and finally Global Average Pooling (GAP) feeding growable fully-connected layers. $C1, C2, C3, C4$ and Z are also free to grow.

possible update v^* for a node should take into account at least all the parameters contributing directly to this node. In Figure 2, when calculating v^* at hidden state $l + 1$, we consider the pre-existing weights W_2 and W_3 .

3.4 When, where, and how much — Strategies level

When it comes to extending the parameters of a network, four questions arise. **When** in the training process should we add new parameters? **Where** exactly in the network should we add these parameters? **How** should these parameters be initialized? And **how much** should we increase the capacity of the network? The question about *how* has been partially answered with the use of the expressivity bottleneck as a guide to optimize the new parameters [11].

In order to answer **when** we should grow, we considered multiple settings where we increase or decrease the number of epochs between each growth step. We conclude that the network can get stuck in local minima, caused by the small parameter size, if trained for too long, echoing similar findings when growing networks in depth [24, 25]. Thus, to make the growth more stable and efficient, the network should preferably be in a non-overfitted state to receive the activation changes introduced by the new parameters. We therefore implement early stopping using a threshold t and a patience p for the validation loss. This gives us a signal of **when** we should grow: whenever the improvement of the validation loss is not sufficient, we stop training and increase the parameters of the network.

The question about **where** we should add new parameters has partially been answered in [12]. The new layer should output directly in the node where the norm of the expressivity bottleneck Ψ is maximal (*restricted strategy*). Several possibilities still remain, and trying all the possible growth steps (quadratic in the number of nodes) to pick the best induces non-negligible costs. In this paper, we attempt to further reduce the number of attempts using the Hilbert-Schmidt Independence Criterion (HSIC). Ideally, we want to compute an estimation of how well a given activity x can be used to fit a two-layer non-linear network (ω, α, b) to obtain an output close to $v_\perp$. Computing the normalized HSIC between the targeted residual and all the preceding activities, we obtain a metric of their alignment. Our choice amounts to choosing the activities x based on their ability to fit a Gaussian kernel that matches $v_\perp$. Although imperfect, this estimate of the ideal new block provides an insight into the true alignment (*HSIC strategy*). In section 4, we conduct an ablation study to analyze the potential benefits of such an estimation and include a strategy where we test all the potential expansions (*greedy strategy*) and a strategy where we choose an action randomly (*random strategy*).

How much we should grow would be easier to answer if a closed-form optimization were available. We can compute the first-order improvement of the new neurons to the bottleneck using the scalar product between δa_l^* and $v_\perp$. If we do the computation separately for each neuron/channel, we have a metric of how much each neuron contributes to the desired direction. Thus, we can select channels based on a threshold s of their first-order improvement. After the optimization is complete, the channels that do not achieve the threshold are discarded.

3.5 DAG Structure and Scheduling — Network level

Since we introduce the notion of growing convolutional networks as arbitrary DAGs, in addition to matching previous node output and current node input size, we now need to also match the spatial dimensions. Our solution is to add padding to all convolutions so as to keep the dimensionality fixed, to allow all nodes to connect with each other. However, this precludes any spatial reduction and would not be memory efficient at scale. Thus, we split our architecture into four independent DAGs, each initialized as a single convolution layer. DAGs are connected sequentially and interleaved with average pooling layers, providing a cumulated dimensionality reduction of a factor 8: see Figure 4. We follow with a global average pooling and two linear layers with a fixed hidden size.

Each DAG grows independently from the others, in a cyclic fashion. When growing DAG1, we have the 3 options described in 3.3, including the growth of the output dimension, $C1$. In fact, growing two layers with a pooling operation in between was not trivial and required major engineering changes on our part, from which we will spare the reader. We cycle through the DAGs by iterating between growth and training steps, always using early stopping: this way, all the parameters up to the first linear layer can be extended.

Adding parameters indefinitely can introduce degenerate solutions where pairs of parameters take large opposite values to trivially minimize the objective. To the best of our knowledge, no regularization scheme fully prevents this type of collapse. For this reason, we implement a stopping criterion for growth. We choose a threshold u for the norm of the desired update v_{∇} at the end of each DAG. If we are below the threshold twice in a row for the same DAG, then we stop growing this DAG forever. When all the DAGs have stopped growing, we execute 100 training epochs and conclude the experiment.

4 Experiments

4.1 Experimental setup

We evaluate Demeter on five datasets from the Unseen Data Challenge [13]. These datasets are intended to assess architecture-design methods beyond standard image-classification benchmarks such as CIFAR-10, where architectures and hyperparameters have been extensively optimized. We employ very minimal hyperparameter tuning to promote frugality and illustrate the quality of the methods when resources are limited. For this reason, we performed grid search only once on MultNIST for the growth-stopping threshold u and the neuron selection threshold s and used the found hyperparameters ($u = 1e^{-5}$, $s = 2.8e^{-4}$) on the rest of the datasets.

We compare three families of methods. First, fixed hand-designed architectures provide standard baselines trained directly on each dataset¹. Second, search-based NAS baselines, including DARTS [7], p-DARTS [8], PC-DARTS [9], and ENAS [6], provide architecture-search baselines. Lastly, our DAG growth-based anytime method. Further details of the experimental setup are given in Appendix B.

Model	Params.
MobilenetV2 [16]	2.3M
MobilenetV3S [17]	1.6M
MobilenetV3L [17]	4.3M
EfficientNetB0 [18]	4.1M
EfficientNetB1 [18]	6.6M
MNasNet [26]	3.2M
ResNet18 [14]	11M
ResNet34 [14]	21M
ResNet50 [14]	23M

Table 1: Number of parameters of fixed-sized models.

4.2 Ablation study - Where to grow

We perform an ablation study of our method to study the impact of selecting where to grow using different strategies. The results are shown in Figure 5 and represent the mean and standard deviation of the methods across 3 metrics: final test accuracy, complexity, and runtime. The *random strategy* is surprisingly not a bad tradeoff between runtime and test accuracy. However, as it utilizes no information from the gradient, it tends to produce much larger models, which also increases the runtime in the long run. The *greedy strategy* has the best overall performance, but is surpassed by the mean of the restricted strategy and is understandably very slow. Overall, the *HSIC strategy* has a very high spread of test accuracy and model complexity, and even though it sometimes surpasses some of the other experiments, its instability is not favorable. The experiments are not long enough for the HSIC to provide a benefit of reduced runtime, either. The most efficient

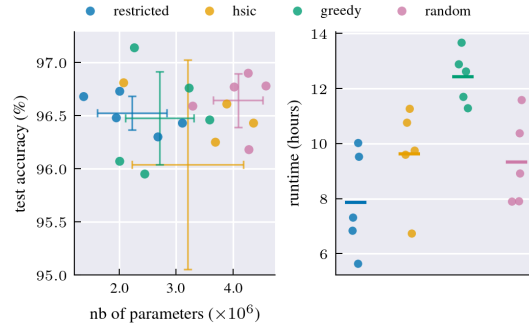


Figure 5: Ablation study of where to grow on MultNIST. The error bars represent the mean and standard deviation over 5 seeds.

¹We unfortunately do not include experiments on [22] because of backward compatibility issues with the requirements' versions in the original implementation.

strategy, taking into account all three metrics, seems to be the *restricted* one. In all the experiments that follow, we will be using the *restricted strategy* for Demeter.

4.3 Benchmarks

We benchmark our method along three axes: test accuracy (Table 3), number of parameters (Table 2), and runtime in GPU hours (Table 4). Demeter starts with the architecture described in Section 3.5 and Figure 4 using 64 initial hidden channels between each DAG and a fixed size of 128 neurons for the last linear layer using the restricted strategy. The output architecture of Demeter is that which maximizes the validation accuracy: our results plot the corresponding test accuracy.

Demeter surpasses all benchmark models and is the only NAS method to always do so (c.f. colors in Table 3 which indicate improvement over benchmark models (in green) or significantly poor performance vs the worst baseline (in red)). It also outperforms ResNet18 in terms of accuracy while using significantly fewer parameters. Interestingly, in this fixed budget setup, ResNet18 always performs better than ResNet50, suggesting that, when considering a diverse range of datasets, simply increasing the number of parameters is not a panacea.

We compare with standard state-of-the-art NAS techniques. DARTS and PC-DARTS perform search for 50 epochs while pDARTS performs 25 epochs per stage, totalling 75 epochs. All DARTS variants train the final supernet from scratch for 200 epochs. ENAS is a reinforcement learning technique, and it becomes quite obvious that the fixed budget of 600 epochs is not enough for the algorithm to converge. We highlight that for most datasets, DARTS and its variants have state-of-the-art performance. Specifically, on CifarTile, which is a dataset closely related to Cifar10, DARTS outperforms all other methods and models by far. We attribute this to the fact that DARTS was designed specifically for CIFAR-type datasets and has been over-engineered to perform well. On all other datasets, Demeter achieves competitive performance, outperforming all methods on Geoclassing and Chesseract. It also acquires the smallest models on MultNIST and CifarTile.

In terms of runtime, Demeter achieves the fastest performance in 3 out of 5 datasets, with all completing in less than 13 GPU hours. It is a well-known issue that DARTS is a slow algorithm, and this is visible on the results in Table 4: Geoclassing requires more than 100 GPU hours, and none of the datasets were completed in less than 14 hours. P-Darts and PC-DARTS are successors of DARTS with reduced computational time. These methods perform quite well on MultNIST, Gutenberg, and Chesseract, but do not scale well with the size of the dataset. On CifarTile and Geoclassing, which are both images of size $64 \times 64 \times 3$, they perform very poorly compared to other methods, both in terms of accuracy and speed. On the contrary, Demeter maintains a stable runtime speed and does not depend on the raw size of the dataset but rather its innate complexity. In Figure 1 we plot the tradeoff between test accuracy (y-axis), runtime (x-axis) and parameter count (marker size) aggregated over all datasets. Overall, Demeter provides a well-balanced tradeoff in terms of accuracy, complexity, and runtime.

In Table 6, we showcase that compared to DARTS, Demeter reduces GPU energy consumption by 77.3% on average across datasets. Demeter’s reduced energy footprint makes it particularly suitable for resource-constrained environments, such as academic labs or edge devices, where large-scale GPU clusters are unavailable.

Method	MultNIST	CifarTile	Gutenberg	Geoclassing	Chesseract
DARTS	3.6M $\pm$ 0.2M	4.1M $\pm$ 0.1M	3.9M $\pm$ 0.2M	1.4M $\pm$ 0.3M	4.5M $\pm$ 0.1M
pDARTS	3.7M $\pm$ 0.4M	3.6M $\pm$ 0.3M	2.9M $\pm$ 42K	2.7M $\pm$ 1M	3.8M $\pm$ 18K
PC-DARTS	3.7M $\pm$ 25K	2.4M $\pm$ 0.9M	1.7M $\pm$ 0.2M	3.2M $\pm$ 0.3M	1.7M $\pm$ 0.2M
Demeter (Ours)	2.2 $\pm$ 0.6M	0.7M $\pm$ 0.2M	4.1M $\pm$ 0.4M	3M $\pm$ 0.6M	9.4M $\pm$ 2.5M

Table 2: Number of Parameters. The numbers represent the mean and standard deviation when relevant. Standard models are trained on 5 seeds and NAS algorithms on 3 seeds.

4.4 Anytime Neural Architecture Search

NAS methods can be classified as many-shot, one-shot, or even zero-shot in terms of the number of training runs required to find a desired target architecture [27]. However, to find highly-performing weights for that architecture, an additional training run is typically required. This introduces a gap

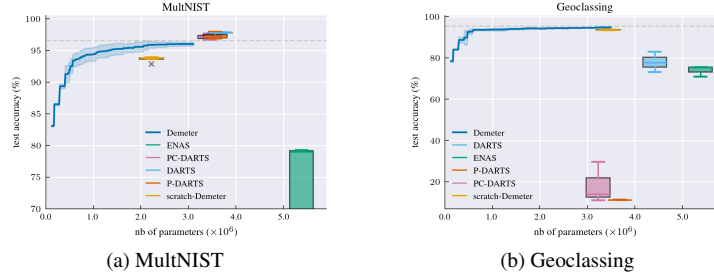


Figure 6: Test accuracy w.r.t. the number of parameters on MultNIST and Geoclassing. Confidence intervals represent the mean and 95% IQL (5 seeds). Boxplots show the accuracy when training the best found architecture from scratch and are positioned on the mean number of parameters (3 seeds).

Categ.	Method	MultNIST	CifarTile	Gutenberg	Geoclassing	Chesseract
Fixed architecture standard optim.	MobilenetV2	87.6% $\pm$ 0.8	43.2% $\pm$ 1	34.2% $\pm$ 0.9	93.6% $\pm$ 0.1	50.2% $\pm$ 0.9
	MobilenetV3S	81.3% $\pm$ 2	39.4% $\pm$ 0.5	31.7% $\pm$ 0.8	91.1% $\pm$ 0.4	53.3% $\pm$ 1
	MobilenetV3L	86.7% $\pm$ 0.8	42.8% $\pm$ 0.5	34.5% $\pm$ 0.2	93.7% $\pm$ 0.2	51.9% $\pm$ 1
	EfficientNetB0	89.7% $\pm$ 1	47% $\pm$ 0.7	35.4% $\pm$ 0.6	94.6% $\pm$ 0.4	51.7% $\pm$ 0.6
	EfficientNetB1	88.4% $\pm$ 0.8	48% $\pm$ 0.7	34.4% $\pm$ 0.6	94.3% $\pm$ 0.2	51.3% $\pm$ 1
	MNASNet	87.6% $\pm$ 0.4	43.1% $\pm$ 0.6	33.4% $\pm$ 0.5	92.5% $\pm$ 0.2	49.3% $\pm$ 1
	ResNet18	94.3% $\pm$ 0.2	42.2% $\pm$ 0.5	40.1% $\pm$ 0.4	92.5% $\pm$ 0.1	53.6% $\pm$ 0.5
	ResNet34	94.6% $\pm$ 0.2	42.2% $\pm$ 0.7	40% $\pm$ 0.4	92.4% $\pm$ 0.5	52.1% $\pm$ 1
NAS SOTA	ResNet50	89.2% $\pm$ 0.7	40.3% $\pm$ 0.7	38.6% $\pm$ 0.5	92.7% $\pm$ 0.5	49.5% $\pm$ 0.4
	DARTS	97.8% $\pm$ 0.1	86% $\pm$ 1	49.9% $\pm$ 1	77% $\pm$ 4	59.6% $\pm$ 0.4
	pDARTS	97.5% $\pm$ 0.4	24.6% $\pm$ 0.2	46.1% $\pm$ 1	11.2% $\pm$ 0.04	59.9% $\pm$ 0.3
	PC-DARTS	97.3% $\pm$ 0.47	55% $\pm$ 3	44% $\pm$ 0.8	18.2% $\pm$ 9	57.5% $\pm$ 0.3
	ENAS	48.8% $\pm$ 23	24% $\pm$ 0.1	24.7% $\pm$ 4	73% $\pm$ 2	57% $\pm$ 0.4
Growth	Demeter (Ours)	96.4% $\pm$ 0.1	55.7% $\pm$ 2.7	41.6% $\pm$ 0.8	95.2% $\pm$ 0.3	56.4% $\pm$ 1.3

Table 3: Test Accuracy selected on the best validation accuracy. The numbers represent the mean and standard deviation. Standard models are trained on 5 seeds and NAS algorithms on 3 seeds. Best scores among each category of methods are shown in **bold**. For NAS/Growth methods, accuracies higher than the best standard model are colored in **green**, while those lower than the worst standard model are colored in **red**.

between the search and evaluation performance. For a previously unseen problem, one cannot know in advance when to stop searching and retrain. Efforts have been made to close the search-optimization gap, for example, P-DARTS [8]. However, these still introduce an arbitrary timescale over which to anneal the architecture towards the final one.

We argue that a practical NAS algorithm should have the property of being “anytime”, independent of some arbitrary stopping time chosen a priori. In this section, we therefore measure the “anytime performance” that one would achieve if search can be stopped arbitrarily, for example, when the user deems the performance to be sufficient or when compute resources run out.

To showcase the concept of anytime-NAS, we compare our method with the fastest DARTS variants, pDARTS and PC-DARTS. We are aware that differential architecture search is not designed to be anytime, and it requires retraining a bigger supernet from scratch after discovering the cells. This is necessary because searching for the cells in the final supernet would be intractable. However, to imitate this behavior, we can track their performance during the search phase and assume that this is the performance we would get at any time.

We acknowledge that runtime as a metric is very sensitive and depends on multiple factors besides the algorithm efficiency, for example, GPU type, availability of resources, batch sizes, etc. Considering that we are not using the same GPU for all experiments, and to make the comparison fair, we standardize the runtime of the methods and just compare their progress until completion. In Figure 7, we can see that the search phases of pDARTS and PC-DARTS do not reach the performance of Demeter, and the Pareto front is dominated by our method. We show that Demeter has a much smaller search-optimization gap, performing well even without re-training, and thus is much more suitable for Anytime NAS.

Method	MultNIST	CifarTile	Gutenberg	Geoclassing	Chessract
DARTS	58.3 ± 6.4	86 ± 10.5	15.7 ± 0.9	101 ± 17.9	14.8 ± 2.2
pDARTS	12.4 ± 3.3	119.9 ± 0.1	11 ± 1.7	151.7 ± 0.1	9.6 ± 1.2
PC-DARTS	20.7 ± 0.6	78 ± 11.4	7.3 ± 1.1	119.8 ± 0.1	6.13 ± 0.2
ENAS	21.9 ± 2.3	104 ± 4.5	24.2 ± 2	108 ± 13.8	19.4 ± 3.3
Demeter (Ours)	7.86 ± 1.8	5.11 ± 0.5	12.8 ± 2.4	16.9 ± 4.2	7.9 ± 2.1

Table 4: Runtime in GPU hours. The numbers represent the mean and standard deviation. Standard models are trained on 5 seeds and NAS algorithms on 3 seeds.

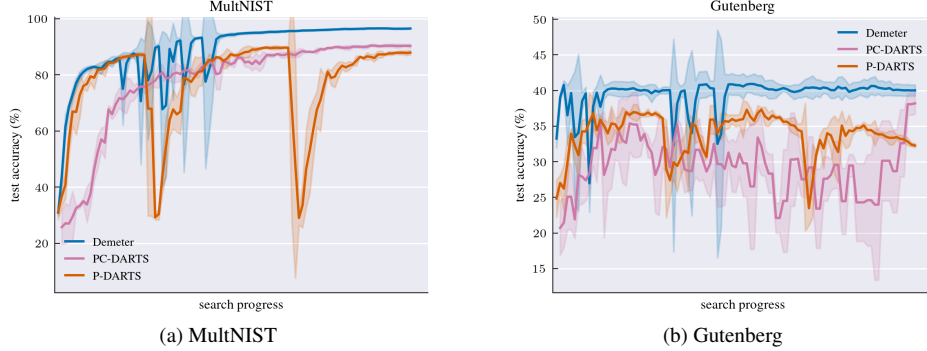


Figure 7: One-shot test accuracy performance during search phases standardized to the length of the experiments. The confidence intervals represent the mean and 95% IQR over 3 seeds.

5 Discussion

DARTS suffers by design from two structural limitations. First, as it selects a subnetwork from a predefined supernet, it cannot build architectures larger than that supernet and thus cannot adapt to very complex tasks. Meanwhile, choosing a larger supernet to be able to deal with such tasks would imply a waste of computational resources for all simple tasks. Therefore, a hyper-parameterization of DARTS makes assumptions on the distributions of tasks and their difficulty, or, otherwise said, DARTS requires task-specific hyper-parameterization when facing more varied tasks. This is confirmed by our experimental results: DARTS may lead by a far margin on standard-looking datasets on which it was originally hyper-tuned (CIFAR-like), but lags behind Demeter on more offbeat datasets such as Geoclassing.

Second, it imposes a particular structure based on a manually-designed tree of cells, and imposes groups of cells to share an identical internal architecture. Our approach is not limited by such arbitrary restrictions. Besides, the architectures we obtain do have different blocks at different resolutions (see Figure 8 in Appendix), which suggests that architectures found by DARTS may be further improved by applying Demeter afterwards.

Limitations The expansion of the DAG introduces a very diverse search space. However, to reduce complexity, we have used only a part of those possibilities. Our method does not introduce convolutions of different kernel sizes in the same DAG for the sake of simplicity and ease of connectivity. We are also not considering different activation functions or skip connections. Also, the seed architecture has been decided beforehand as justified in Section 3.5.

6 Conclusion

Our method is the first, to the best of our knowledge, to start from a very small seed network for visual tasks. Demeter manages to achieve SOTA or near-SOTA performance while starting in the under-parametrized regime, and ending up in the over-parametrized one (most likely). In terms of theory of learning dynamics, this offers a counter-example to the now widely accepted belief that over-parametrization is necessary for efficiently training deep networks. Our initialization of new weights to an optimal state probably plays a key role there. We hope these questions will pave the way for future theoretical investigation. All in all, we show that Demeter provides a well-balanced tradeoff in terms of accuracy, complexity, and runtime with a significantly reduced energy footprint.

References

- [1] Barret Zoph and Quoc V. Le. Neural architecture search with reinforcement learning. *CoRR*, abs/1611.01578, 2016. URL <http://arxiv.org/abs/1611.01578>.
- [2] Esteban Real, Sherry Moore, Andrew Selle, Saurabh Saxena, Yutaka Leon Suematsu, Jie Tan, Quoc V. Le, and Alexey Kurakin. Large-scale evolution of image classifiers. In Doina Precup and Yee Whye Teh, editors, *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 2902–2911. PMLR, 06–11 Aug 2017. URL <https://proceedings.mlr.press/v70/real17a.html>.
- [3] Esteban Real, Alok Aggarwal, Yanping Huang, and Quoc V. Le. Regularized evolution for image classifier architecture search. *CoRR*, abs/1802.01548, 2018. URL <http://arxiv.org/abs/1802.01548>.
- [4] Barret Zoph, Vijay Vasudevan, Jonathon Shlens, and Quoc V. Le. Learning transferable architectures for scalable image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2018.
- [5] Yawen Duan, Xin Chen, Hang Xu, Zewei Chen, Xiaodan Liang, Tong Zhang, and Zhenguo Li. TransNAS-Bench-101: Improving Transferability and Generalizability of Cross-Task Neural Architecture Search, May 2021. URL <http://arxiv.org/abs/2105.11871>. arXiv:2105.11871 [cs].
- [6] Hieu Pham, Melody Guan, Barret Zoph, Quoc Le, and Jeff Dean. Efficient neural architecture search via parameters sharing. In Jennifer Dy and Andreas Krause, editors, *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 4095–4104. PMLR, 10–15 Jul 2018. URL <https://proceedings.mlr.press/v80/pham18a.html>.
- [7] Hanxiao Liu, Karen Simonyan, and Yiming Yang. DARTS: differentiable architecture search. *CoRR*, abs/1806.09055, 2018. URL <http://arxiv.org/abs/1806.09055>.
- [8] Xin Chen, Lingxi Xie, Jun Wu, and Qi Tian. Progressive differentiable architecture search: Bridging the depth gap between search and evaluation. *CoRR*, abs/1904.12760, 2019. URL <http://arxiv.org/abs/1904.12760>.
- [9] Yuhui Xu, Lingxi Xie, Xiaopeng Zhang, Xin Chen, Guo-Jun Qi, Qi Tian, and Hongkai Xiong. PC-DARTS: partial channel connections for memory-efficient differentiable architecture search. *CoRR*, abs/1907.05737, 2019. URL <http://arxiv.org/abs/1907.05737>.
- [10] Joseph Mellor, Jack Turner, Amos Storkey, and Elliot J. Crowley. Neural Architecture Search without Training, June 2021. URL <http://arxiv.org/abs/2006.04647>. arXiv:2006.04647 [cs].
- [11] Manon Verbockhaven, Théo Rudkiewicz, Sylvain Chevallier, and Guillaume Charpiat. Growing tiny networks: Spotting expressivity bottlenecks and fixing them optimally. *Transactions on Machine Learning Research*, 2024. URL <https://openreview.net/forum?id=hbtG6s6e7r>.
- [12] Stella Douka, Manon Verbockhaven, Théo Rudkiewicz, Stephane Rivaud, Franccois P. Landes, Sylvain Chevallier, and Guillaume Charpiat. Growth strategies for arbitrary dag neural architectures. In *ESANN*, 2025. URL <https://api.semanticscholar.org/CorpusID:275787923>.
- [13] Rob Geada, David Towers, Matthew Forshaw, Amir Atapour-Abarghouei, and A. Stephen McGough. Insights from the use of previously unseen neural architecture search datasets. In *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 22541–22550, 2024. doi: 10.1109/CVPR52733.2024.02127.
- [14] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep Residual Learning for Image Recognition, December 2015. URL <http://arxiv.org/abs/1512.03385>. arXiv:1512.03385 [cs].
- [15] Sergey Zagoruyko and Nikos Komodakis. Wide Residual Networks, June 2017. URL <http://arxiv.org/abs/1605.07146>. arXiv:1605.07146 [cs].
- [16] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. MobileNetV2: Inverted Residuals and Linear Bottlenecks, March 2019. URL <http://arxiv.org/abs/1801.04381>. arXiv:1801.04381 [cs].
- [17] Andrew Howard, Mark Sandler, Grace Chu, Liang-Chieh Chen, Bo Chen, Mingxing Tan, Weijun Wang, Yukun Zhu, Ruoming Pang, Vijay Vasudevan, Quoc V. Le, and Hartwig Adam. Searching for MobileNetV3, November 2019. URL <http://arxiv.org/abs/1905.02244>. arXiv:1905.02244 [cs].

- [18] Mingxing Tan and Quoc Le. EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. In *Proceedings of the 36th International Conference on Machine Learning*, pages 6105–6114. PMLR, May 2019. URL <https://proceedings.mlr.press/v97/tan19a.html>.
- [19] Thomas Elsken, Jan Hendrik Metzen, and Frank Hutter. Efficient multi-objective neural architecture search via lamarckian evolution. 2019. URL <https://arxiv.org/abs/1804.09081>.
- [20] Utku Evci, Bart van Merriënboer, Thomas Unterthiner, Fabian Pedregosa, and Max Vladymyrov. GradMax: Growing Neural Networks using Gradient Information. In *International Conference on Learning Representations*, 2022. URL https://openreview.net/forum?id=qjN4h_wwU0.
- [21] Tianqi Chen, Ian Goodfellow, and Jonathon Shlens. Net2Net: Accelerating Learning via Knowledge Transfer. In *International Conference on Learning Representations*, 2016. doi: 10.48550/arXiv.1511.05641. URL <http://arxiv.org/abs/1511.05641>. arXiv:1511.05641 [cs].
- [22] Rupert Mitchell, Robin Menzenbach, Kristian Kersting, and Martin Mundt. Self-Expanding Neural Networks, 2024. URL <http://arxiv.org/abs/2307.04526>. arXiv:2307.04526 [cs].
- [23] Thomas Elsken, Jan Hendrik Metzen, and Frank Hutter. Efficient multi-objective neural architecture search via lamarckian evolution, 2019. URL <https://arxiv.org/abs/1804.09081>.
- [24] Wei Wen, Feng Yan, Yiran Chen, and Hai Li. AutoGrow: Automatic Layer Growing in Deep Convolutional Networks, June 2020. URL <http://arxiv.org/abs/1906.02909>. arXiv:1906.02909 [cs].
- [25] Haihang Wu, Wei Wang, Tamasha Malepathirana, Damith Senanayake, Denny Oetomo, and Saman Halgamuge. When to grow? a fitting risk-aware policy for layer growing in deep neural networks. In *Proceedings of the Thirty-Eighth AAAI Conference on Artificial Intelligence and Thirty-Sixth Conference on Innovative Applications of Artificial Intelligence and Fourteenth Symposium on Educational Advances in Artificial Intelligence*, AAAI’24/IAAI’24/EAAI’24. AAAI Press, 2024. ISBN 978-1-57735-887-9. doi: 10.1609/aaai.v38i6.28414. URL <https://doi.org/10.1609/aaai.v38i6.28414>.
- [26] Mingxing Tan, Bo Chen, Ruoming Pang, Vijay Vasudevan, Mark Sandler, Andrew Howard, and Quoc V. Le. MnasNet: Platform-Aware Neural Architecture Search for Mobile, May 2019. URL <http://arxiv.org/abs/1807.11626>. arXiv:1807.11626 [cs].
- [27] Thomas Elsken, Jan Hendrik Metzen, and Frank Hutter. Neural architecture search: A survey. *Journal of Machine Learning Research*, 20(55):1–21, 2019.
- [28] Arthur Gretton, Kenji Fukumizu, Choon Teo, Le Song, Bernhard Schölkopf, and Alex Smola. A kernel statistical test of independence. In J. Platt, D. Koller, Y. Singer, and S. Roweis, editors, *Advances in Neural Information Processing Systems*, volume 20. Curran Associates, Inc., 2007. URL https://proceedings.neurips.cc/paper_files/paper/2007/file/d5cfead94f5350c12c322b5b664544c1-Paper.pdf.

A Architecture

Our seed architecture starts with only one convolutional layer with 64 channels per DAG. All convolutions have kernels of size 3×3 and are padded to preserve dimensionality. The hidden size of the first linear layer is fixed at 128 neurons. All nodes are followed by SeLU activation functions and a custom growable LayerNorm. When we have parallel convolutions, only one of them uses bias.

Method	batch size	MultNIST
pDARTS	128	8.50 ± 1.7
Demeter	256	3.09 ± 0.4

Table 5: Maximum GPU memory allocation. The numbers represent the mean and standard deviation.

Method	MultNIST	CifarTile	Gutenberg	Geoclassing	Chessract
DARTS	9.966 ± 0.844	14.144 ± 2.977	3.072 ± 0.205	15.036 ± 4.097	1.189 ± 0.439
Demeter	0.752 ± 0.282	0.750 ± 0.068	0.680 ± 0.065	3.016 ± 0.685	0.696 ± 0.392

Table 6: GPU consumption (kWh). The numbers represent the mean and standard deviation over 3 seeds. MultNIST is using 5 seeds.

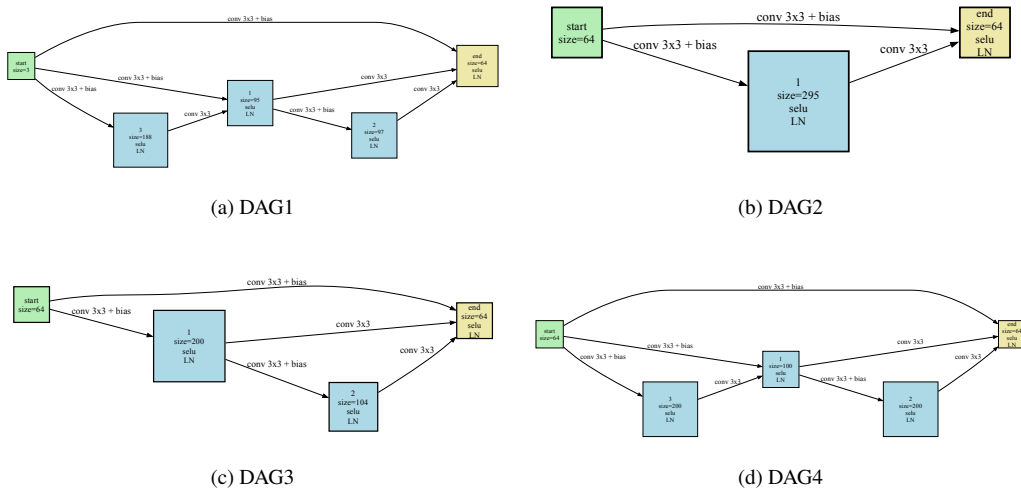


Figure 8: Derived architecture of the best performing model on MultNIST

Dataset	Batch size
MultNIST	128 (56 for DARTS)
Cifar10	16
Gutenberg	128
Geoclassing	16
Chessract	128

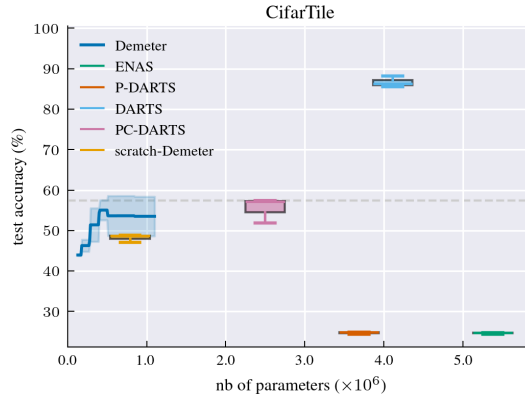
Table 7: batch sizes used for DARTS variants.

B Hyperparameters

Demeter, during the train phases, is trained using AdamW, learning rate 10^{-3} , weight decay $8e^{-3}$, eps $1e^{-4}$ to promote stability, and a cosine learning-rate schedule over a maximum of 100 epoch with eta_min $1e^{-5}$. We are using gradient clipping and early stopping on the validation loss when it is below the threshold $t = 1.22e^{-3}$ for 3 consecutive epochs. For this stage, we use a batch size of 256 for all experiments. During the growth phases, 100 new neurons are optimized using AdamW, learning rate $1e^{-3}$ over 100 epochs with a batch size of 256. The neurons are then selected using the threshold $s = 2.8e^{-4}$ on their first order improvement. The expressivity bottleneck is calculated by iterating through 2048 samples of the optimization dataset using a batch size of 64. When using the HSIC strategy, we use 1000 samples of the optimization dataset to calculate it. We stop growing a DAG when the norm of the desired update v_{∇} at its output is below the threshold $u = 1e^{-5}$ for 2 consecutive growth steps.

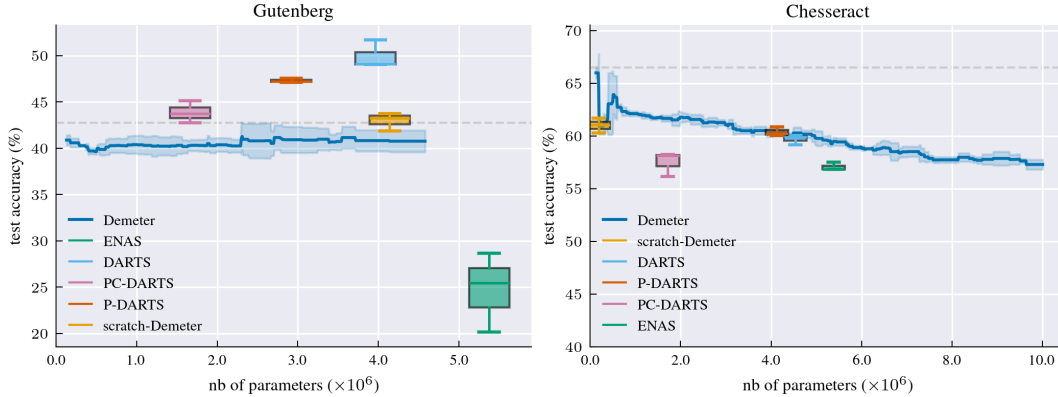
Fixed architectures are trained from scratch with SGD, learning rate 10^{-2} , momentum 0.9, weight decay $1e^{-4}$, and a cosine learning-rate schedule over 150 epochs, including 10 warmup epochs. We use a batch size of 64 for all experiments. DARTS and PC-DARTS are executing 50 search steps and 200 train steps, while pDARTS is executing 25 search steps per stage. Due to memory limitations, for these methods, we have used different batch sizes for each dataset presented in Table 7. For the rest of the hyperparameters, we used the default values reported in the original implementation.

Benchmark results for fixed architectures are reported over five random seeds, while NAS baselines and Demeter are reported over three random seeds. The MultNIST growth-strategy ablation is reported separately over five random seeds.



(a) CifarTile

Figure 9: Maximum test accuracy achieved over the number of parameters. The confidence intervals represent the mean and 95% IQL over 3 seeds. The boxplots show the accuracy when training the best found architecture from scratch and are positioned on the mean number of parameters over 3 seeds.



(a) Gutenberg

(b) Chesseract

Figure 10: Maximum test accuracy achieved over the number of parameters. The confidence intervals represent the mean and 95% IQL over 3 seeds. The boxplots show the accuracy when training the best found architecture from scratch and are positioned on the mean number of parameters over 3 seeds.

C Data and Augmentation

We use 10% of our training dataset as validation. The rest of the data is further split and shuffled, at each growth step, into 60% optimization and 40% development. We use the optimization set to optimize the new weights with gradient descent and the development set to perform line search for the scaling factor of the new weights. The validation split is always fixed and is used to determine which expansion to apply and for early stopping of the training phases. The training phases are using both the optimization and the development sets.

Due to the variability and unique nature of the datasets, we paid very close attention to the augmentation techniques we could use. For example, a mindless rotation of datasets like Chesseract or Gutenberg, where the data is structured in sparse matrices, could be detrimental. Specifically, for MultNIST, we use a RandomAffine transformation applied separately per channel. For CifarTile and Geoclassing, we use RandomHorizontalFlip. Geoclassing also applies RandomVerticalFlip, random Rotation between 0, 90, 180, and 270 degrees, and RandomErasing. Gutenberg and Chesseract do not use any augmentation.

In some of the datasets, especially Chesseract, our method overfits to the train and validation sets as the number of parameters increases. In fact, we notice this trend on Chesseract in all the methods. This can be attributed either to our choice of splitting the datasets in a specific way. We believe this is due to the fact that in Chesseract, the train/validation splits come from a relatively small number of

501 games, including multiple sequential board positions from the same games, while the test set contains
502 an entirely independent set of games. Therefore, all of the methods can memorise the train-set game
503 board positions rather than generalising. This is also related to the variation presented in Figure 7
504 during the growth procedure. We notice performance drops immediately after the introduction of new
505 parameters. Ultimately, the choice of new parameters depends on the loss of the validation set, which
506 is monotonically decreasing, but is not fully correlated with the test accuracy that we present in the
507 plots. This issue can also be related to excessive training during the under-parameterized setting that
508 pushes the parameters into local minima. A solution would be to change this criterion and use the
509 validation accuracy directly, consider a different separation of the data, or be more strict about early
510 stopping at the beginning of the process. We intend to investigate this instability further for future
511 work.

512 **D Method Details**

513 **D.1 Adding layers to the DAG**

One can consider a neural network as a computational graph structured as a directed acyclic graph (DAG). In this graph, each edge represents a layer, or else a weight tensor, and each node represents a hidden state in the network. Multiple layers can output to the same hidden state and then be merged using a specific operation, usually addition or concatenation. Choosing to use addition as a merge operation requires that all activities have the same output dimension. Subsequently, all layers outputting to the same hidden state should have the same number of output features. In Figure 2, we present an example of such a DAG where all pre-activities are denoted as A and all post-activities are denoted as B_i or x . The peculiarity of the DAG is that while computing the optimal updates v^* on the hidden state $l + 1$, we need to take into account all pre-existing weights W_2 and W_3 . So our expressivity bottleneck is defined as

$$v_{\perp} := v_{\nabla} - \delta W_2^* - \delta W_3^*$$

514 At any given time, we can expand the DAG in 3 possible ways:

- 515 • Increase the number of neurons of node i . This requires increasing the output dimension
516 of all preceding layers and the input dimension of all succeeding layers. For example, to
517 increase the number of neurons of node 2 in Figure 2 by k neurons, one should add k new
518 rows to each of the two previous layers and k new columns to the next layer.
- 519 • Introduce a new node to the graph between two specific nodes. This requires creating two
520 new layers α and ω , with an arbitrary number of neurons k . The input dimension of layer
521 α should match the output size of the previous node, and the output dimension of layer ω
522 should match the input size of the next node.
- 523 • Add a single layer α connecting two pre-existing nodes. The input dimension of the new
524 layer should match the output size of the previous node, and the output dimension should
525 match the input size of the next node.

526 In the first two cases, we can use the same optimization scheme as described in Section ??, no matter
527 if α and ω represent a whole weight matrix or only part of it. For the addition of a single layer, we
528 simply define $\delta\theta_l^* = \alpha * x + b$.

529 **D.1.1 Engineering**

530 Here we discuss some important implementation issues that arise when dealing with growing convo-
531 lution networks. The hidden nodes of the network are contained in what we call ‘Merge’ modules,
532 for reasons that we now explain.

533 The nodes in the DAG can be considered ‘Merge’ modules that facilitate the storage of activities and
534 inputs in the network during the forward and backward processes. This avoids storing them multiple
535 times in different modules. The ‘Merge’ modules can also keep track of the modules that precede
536 or succeed them and apply activation functions to their combined output. They are also responsible
537 for computing the optimal updates and the expressivity bottleneck for all the modules they merge
538 collectively. They have access to the full combined input from all the previous modules at once and
539 can provide information that is informed by all paths.

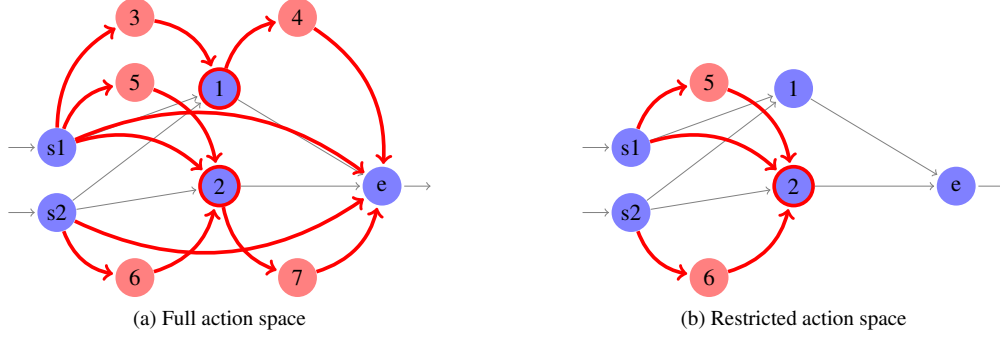


Figure 11: Search space visualization

When extending to convolutions, these modules also need the ability to reason for the whole block of input and output layers as one to facilitate growth. That being said, the ‘Merge’ module can repackage its output into a matrix of patch vectors, one per spatial location per sample, so that analogous computations to the fully-connected case can be applied by additionally summing over the patch dimension. Instead of simply using the two-dimensional input in the linear case, here we need to use the unfolded input and combine it per sample per patch.

A further complication arises when pooling operations are interleaved between growing modules. Spatial downsampling changes the resolution of the feature maps and, therefore, the shape of the statistics tensors [11] that the growing procedure relies on. Supporting growth across such transitions is facilitated by enforcing that two ‘Merge’ modules appear in direct succession, with the pooling operation encapsulated within the first one. This pairing is essential because the growth computation propagates backwards through the graph: each ‘Merge’ module is responsible for updating the statistics of its predecessors, and when it encounters another ‘Merge’ module immediately before it, it can safely delegate that module’s statistical accounting to the first module itself, avoiding double-counting across the boundary. The non-linearity applied by the first ‘Merge’ module is naturally captured through its activity, and any other contributions flowing out of it are accounted for via the gradient of its pre-activity. This clean separation means the second ‘Merge’ module needs only to handle any additional inputs of its own, while both modules retain statistics tensors at the correct dimensionality for their respective sides of the downsampling operation.

The same consecutive-‘Merge’ structure enables growth across the transition from convolutional to fully-connected layers via global average pooling, where the spatial aggregation is again encapsulated within the first ‘Merge’ module. Any flattening of the resulting representation can then be placed freely in the forward pass between the two modules without disrupting the statistics: the first module stores its activity before the flattening occurs, preserving the spatial structure needed for the patch-wise unfolding on the convolutional side, while the second module operates on the already-flattened representation and accumulates its statistics tensors in the linear regime. The optimal updates for each side are therefore computed from statistics that reflect the correct tensor geometry on that side of the transition.

Together, these properties allow the ‘Merge’ modules to handle expansion across dimensionality shifts and operation types, while naturally supporting early exits or additional inputs at any point in the network.

D.2 Search Space Definition

We define the search space as a DAG with nodes $\mathcal{N} = \{0, 1, \dots, N - 1\}$, where each node n_j represents a feature map. An action is a tuple $a = (i, j, o) \in \mathcal{A} \subseteq \mathcal{N} \times \mathcal{N} \times \mathcal{O}$, subject to $i < j$, specifying that operation o is applied to n_i and its output added to n_j :

$$n_j = \sum_{(i,j,o) \in \mathcal{A}_j} o(n_i), \quad \mathcal{A}_j = \{(i, j', o) \in \mathcal{A} : j' = j\}$$

Using Figure 11a as an example, we can consider a seed network with intermediate nodes 1 & 2 (in blue color). The starting and end nodes are denoted as s_i and e , respectively. Then some of the possible actions we can perform are visible in red. All the possible actions we can perform are:

- | | |
|--|--|
| 575 • Add direct edge from $s1$ to e | 580 • Add new node 5 from $s1$ to 2 |
| 576 • Add direct edge from $s1$ to 2 | 581 • Add new node 6 from $s2$ to 2 |
| 577 • Add direct edge from $s2$ to e | 582 • Add new node 7 from 2 to e |
| 578 • Add new node 3 from $s1$ to 1 | 583 • Increase the hidden size of node 1 |
| 579 • Add new node 4 from 1 to e | 584 • Increase the hidden size of node 2 |

The simplest option is to employ a greedy strategy $\text{greedy}(\mathcal{A})$ in which we test and optimize the weights of all possible actions $\mathcal{A}$ at once. However, trying all the possible growth steps (quadratic in the number of nodes) to pick the best one induces non-negligible costs. Thus, we should find a valid way to reduce the action space. A simple solution is to follow the random strategy $\text{random}(\mathcal{A})$ where, as the name implies, we choose an action at random and execute it. Another option is to follow a strategy informed by the expressivity bottleneck norm Ψ . In this restricted strategy, the new layer should output directly at the node where the norm of the bottleneck is maximized

$$j^* = \arg \max_{j \in \mathcal{N}} \Psi_j$$

585 or directly increase its size. Thus, we define the restricted strategy as $\text{restricted}(\mathcal{A}) = \mathcal{A}_{j^*}$. In
 586 practice we use $\text{greedy}(\text{restricted}(\mathcal{A}))$. In the example in Figure 11b, we can see that if $j^* = 2$ then
 587 the action space is greatly reduced and the actions we need to evaluate are only 4 (shown in red).
 588 While this significantly restricts the action space with respect to the output node, we can go a step
 589 further. We attempt to further reduce the complexity of choosing an action using the Hilbert-Schmidt
 590 Independence Criterion (HSIC) [28]. HSIC is a kernel-based approach to detecting dependence
 591 between random variables present on different metric spaces. We present it in Lemma 1.

Lemma 1 (HSIC) *Given random variables $X = [x_1, x_2, \dots, x_n] \in \mathbb{R}^{n \times d}$ and $Y = [y_1, y_2, \dots, y_n] \in \mathbb{R}^{n \times o}$, we can measure their statistical dependence without parametric assumptions on their joint distribution P_{XY} . Let $k(x, x')$ and $l(y, y')$ denote positive-definite kernels, usually linear or Gaussian. We first form $K, L \in \mathbb{R}^{n \times n}$ with $K_{ij} = k(x_i, x_j)$ and $L_{ij} = l(y_i, y_j)$, at cost $\mathcal{O}(n^2)$. Using the centering matrix $H = I_n - \frac{1}{n} \mathbf{1}\mathbf{1}^\top$, we obtain $\tilde{K} = HKH$ and $\tilde{L} = HLH$, which subtracts the empirical mean embeddings in the induced reproducing kernel Hilbert spaces (RKHS). The estimator is then*

$$\text{HSIC}(X, Y) = \frac{1}{(n-1)^2} \text{Tr}(\tilde{K}\tilde{L})$$

592 *corresponding to the squared Hilbert-Schmidt norm of the empirical cross-covariance operator*
 593 *between the feature embeddings.*

594 Since raw HSIC is not scale-invariant and its magnitude depends on the kernel and marginals, values
 595 across different variable pairs are not directly comparable. We therefore use a normalized variant

$$\text{nHSIC}(X, Y) = \frac{\text{HSIC}(X, Y)}{\sqrt{\text{HSIC}(X, X) \text{HSIC}(Y, Y)}} \in [0, 1] \quad (4)$$

596 analogous to a correlation coefficient in the RKHS, with 0 indicating independence (for characteristic
 597 kernels) and 1 maximal dependence. This renders the criterion comparable across kernels and scales,
 598 and well-suited to ranking-based applications such as feature selection and association analysis
 599 without the need for a significance threshold.

Ideally, we want to compute an estimation of how well a given activity x can be used to fit a two-layer non-linear network (ω, α, b) to obtain an output close to $v_\perp$. By computing the normalized HSIC in Equation 4 between the maximum expressivity bottleneck at the optimal output node and all preceding activities, we obtain a metric of their alignment. In this case, the activities x are chosen based on their ability to fit a Gaussian kernel that matches $v_\perp$. For the Gaussian kernels $K_{ij} = L_{ij} = \exp \frac{-\|x_i - x_j\|^2}{2\sigma^2}$ we set σ via the median heuristic. Even though this is not a perfect estimation of our block, we assume that it is close enough to provide an insight into the true alignment (HSIC strategy). Thus, we define the strategy as

$$\text{hsic}(\mathcal{A}) = \mathcal{A}_{i^*}, \text{ where } i^* = \arg \max_{i \in \mathcal{N}} \text{nHSIC}(X_i, \Psi_j)$$

600 In practice, we use $\text{greedy}(\text{hsic}(\text{restricted}(\mathcal{A})))$. In section 4, we conduct an ablation study to
 601 analyse the potential benefits of such estimation.

Algorithm 1 Growth Step

Notation:

s current network state

$a = (i, j, o)$ action adding operation o from node i to node j

α, b, ω parameters of the new operation, with output $\omega \sigma(\alpha \cdot x + b)$

γ scalar amplitude factor, yielding $\gamma \omega \sigma(\gamma \alpha \cdot x + b)$

$T(s, a)$ state transition applying action a with its optimized parameters

τ threshold on per-channel relevance for selecting which channels to keep

Require: $G = (V, E)$ with $V = \{n_1, \dots, n_N\}$ and $E \subseteq V \times V$ such that $(n_i, n_j) \in E \Rightarrow i < j$

Require: Threshold τ on the per-channel relevance

- 1: $\mathcal{A}(G) \leftarrow \{a : a \text{ is a valid growth action on } G\}$ ▷ Find all valid actions
 - 2: **for all** $n \in V$ **do** ▷ Compute statistics on all nodes
 - 3: $v_{\nabla n} \leftarrow \text{DESIREDUPDATE}(n)$
 - 4: $v_n^* \leftarrow \text{POSSIBLEUPDATE}(n)$
 - 5: $v_{\perp n} \leftarrow v_{\nabla n} - v_n^*$
 - 6: $\Psi_n \leftarrow \|v_{\perp n}\|$
 - 7: **end for**
 - 8: $j^* \leftarrow \arg \max_{j \in V} \Psi_j$ ▷ Find most important node j^*
 - 9: $\tilde{\mathcal{A}} \leftarrow \text{restricted}(\mathcal{A}(G))$ ▷ Restrict action space [*where*]
 - 10: **for all** $a = (i, j, o) \in \tilde{\mathcal{A}}$ **do**
 - 11: Define $\delta\theta_j \leftarrow \omega \sigma(\alpha a_{l-1}(x) + b)$
 - 12: Optimize $\alpha^*, b^*, \omega^* \leftarrow \arg \max_{\alpha, b, \omega} \langle \delta\theta_j, v_{\perp j} / \|v_{\perp j}\| \rangle$ ▷ [*how*]
 - 13: $\{r_c\}_c \leftarrow \{\langle \delta\theta_j^{(c)}, v_{\perp j} / \|v_{\perp j}\| \rangle\}_c$ ▷ Per-channel scalar product
 - 14: $\mathcal{C}^* \leftarrow \{c : r_c \geq \tau\}$ ▷ Keep channels above threshold τ
 - 15: Prune $\delta\theta_j$ to channels in $\mathcal{C}^*$ ▷ [*how much*]
 - 16: Define $s'_\gamma \leftarrow T(s, a \mid \alpha^*, b^*, \omega^*, \gamma)$ ▷ State as a function of amplitude γ
 - 17: Optimize $\gamma^* \leftarrow \arg \min_{\gamma} \mathcal{L}(s'_\gamma \mid \mathcal{D}_{\text{dev}})$ ▷ Compute amplitude factor
 - 18: Evaluate $\ell(a) \leftarrow \mathcal{L}(s_{\gamma^*}' \mid \mathcal{D}_{\text{val}})$
 - 19: **end for**
 - 20: $a^* \leftarrow \arg \min_{a \in \tilde{\mathcal{A}}} \ell(a)$ ▷ Select best action
 - 21: $s \leftarrow T(s, a^*)$ ▷ Transition to new state
-

Algorithm 2 Demeter

Require: Network $\mathcal{N}$ consisting of multiple directed acyclic graphs $g = (V_g, E_g) \in G$
Require: $V_g = \{n_1^g, \dots, n_N^g\}$ and $E_g \subseteq V_g \times V_g$ such that $(n_i^g, n_j^g) \in E_g \Rightarrow i < j$
Require: Threshold u on the desired update norm
Require: Threshold t on the validation loss improvement
Require: Patience p on the early-stopping criterion

- 1: **for all** $g \in G$ **do**
- 2: $\text{strikes}_g \leftarrow 0$
- 3: **end for**
- 4: **while** $\exists g \in G$ such that $\text{strikes}_g < 2$ **do**
- 5: **for all** $g \in G$ such that $\text{strikes}_g < 2$ **do**
- 6: Train $\mathcal{N}$ with early-stopping threshold t and patience p $\triangleright$ Train Step [*when*]
- 7: $v_{\nabla g} \leftarrow \|\text{DESIREDUPDATE}(n_N^g)\|$ $\triangleright$ Desired update norm at end node
- 8: **if** $v_{\nabla g} < u$ **then**
- 9: $\text{strikes}_g \leftarrow \text{strikes}_g + 1$
- 10: **else**
- 11: $\text{strikes}_g \leftarrow 0$
- 12: **end if**
- 13: **if** $\text{strikes}_g < 2$ **then** $\triangleright$ Freeze immediately on reaching 2 strikes
- 14: $g \leftarrow \text{GROWTH STEP}(g)$ $\triangleright$ Growth Step
- 15: **end if**
- 16: **end for**
- 17: **end while**
- 18: Train $\mathcal{N}$ for 100 epochs $\triangleright$ Final fine-tuning

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: The abstract claims that our method is anytime, which is reflected in the Experiments section, and the competitiveness of accuracy is demonstrated on multiple datasets.

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: The limitations have been included and discussed.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper does not include theoretical results.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: All training details are available in the Experiments section and/or the Appendix, and the complete code can be provided after acceptance.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: The code will be open-sourced

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: All training and hyperparameter details are available in the Experiments section and/or the Appendix

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: All tables and figures provide standard deviation and confidence intervals. The number of seeds for each experiment is mentioned in the Experiments section.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).

- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: Runtime and resources are some of our main metrics. They are available either in the Experiments section or the Appendix.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: We are not aware of any immediate violation of the NeurIPS Code of Ethics.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [N/A]

Justification: We are not aware of an immediate negative societal impact of our work.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.

- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The paper poses no such risks

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: All the models, algorithms, papers, and datasets are properly credited. All the datasets use the license CC-BY-4.0

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.

- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [N/A]

Justification: The paper does not release new assets.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The paper does not involve crowdsourcing nor research with human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: The paper does not involve crowdsourcing nor research with human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.

- 910 • We recognize that the procedures for this may vary significantly between institutions
911 and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the
912 guidelines for their institution.
913 • For initial submissions, do not include any information that would break anonymity (if
914 applicable), such as the institution conducting the review.

915 **16. Declaration of LLM usage**

916 Question: Does the paper describe the usage of LLMs if it is an important, original, or
917 non-standard component of the core methods in this research? Note that if the LLM is used
918 only for writing, editing, or formatting purposes and does *not* impact the core methodology,
919 scientific rigor, or originality of the research, declaration is not required.

920 Answer: [N/A]

921 Justification: The core method development in this research does not involve LLMs as any
922 important, original, or non-standard components.

923 Guidelines:

- 924 • The answer [N/A] means that the core method development in this research does not
925 involve LLMs as any important, original, or non-standard components.
926 • Please refer to our LLM policy in the NeurIPS handbook for what should or should not
927 be described.